

CO4 – ASSESSMENT TOOL 3: COMPARATIVE ANALYSIS

Database Management Systems (CSA05) — MySQL Implementation

CO4: File Organization, Indexing & Hashing Techniques | Weightage: 10%

All comparative evaluations below are implemented using formal MySQL relational schemas, analytical stored procedures, and structured tabular queries per the laboratory formatting standard.

PRELIMINARY SETUP – Database & Schema Creation

```
-- Create and select working database
CREATE DATABASE IF NOT EXISTS dbms_at3;
USE dbms_at3;

CREATE TABLE comparison_records (
    comp_id      INT AUTO_INCREMENT PRIMARY KEY,
    topic        VARCHAR(80),
    parameter    VARCHAR(50),
    technique_a  VARCHAR(100),
    technique_b  VARCHAR(100),
    tradeoff     VARCHAR(150)
);
```

Question 1

Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

```
USE dbms_at3;

CREATE TABLE comp_q1 (
    operation_aspect  VARCHAR(30) PRIMARY KEY,
    heap_file_org     VARCHAR(60),
    sequential_file_org VARCHAR(60),
    tradeoff_summary  VARCHAR(80)
);

INSERT INTO comp_q1 VALUES
('Insertion Cost',      'O(1) - Append to last block',      'O(B) - Requires sorting & block shifting', 'Heap is optimal for heavy ingestion'),
('Deletion Cost',      'O(1) - Logical tombstone deletion',  'O(B) - Physical rewrite / merge',      'Heap avoids costly disk defragmentation'),
('Point Retrieval',    'O(B) - Linear exhaustive scan',      'O(log2 B) - Binary search supported',      'Sequential is significantly faster'),
('Range Retrieval',    'O(B) - Must scan all data blocks',      'O(log2 B + k) - Direct contiguous traversal', 'Sequential excels at ordered scans');

SELECT * FROM comp_q1;
```

OUTPUT / TERMINAL RESULT:

```
mysql>
mysql> SELECT * FROM comp_q1;
```

operation_aspect	heap_file_org	sequential_file_org	tradeoff_summary
Deletion Cost	O(1) - Logical tombstone deletion	O(B) - Physical rewrite / merge	Heap avoids costly disk defragmentation
Insertion Cost	O(1) - Append to last block	O(B) - Requires sorting & block shifting	Heap is optimal for heavy ingestion
Point Retrieval	O(B) - Linear exhaustive scan	O(log2 B) - Binary search supported	Sequential is significantly faster
Range Retrieval	O(B) - Must scan all data blocks	O(log2 B + k) - Direct contiguous traversal	Sequential excels at ordered scans

```
"=====
```

Question 2

Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

```
USE dbms_at3;

CREATE TABLE comp_q2 (
    aspect          VARCHAR(30) PRIMARY KEY,
    clustered_index  VARCHAR(60),
    non_clustered_index VARCHAR(60),
    industry_example VARCHAR(80)
);

INSERT INTO comp_q2 VALUES
('Physical Storage', 'Dictates physical record order on disk', 'Stores key + pointer; data order unchanged', 'InnoDB PRIMARY KEY vs secondary index'),
('Count per Table', 'Strictly 1 per table', 'Multiple secondary indexes allowed', '1 Clustered vs N Non-Clustered'),
('Leaf Node Content', 'Actual complete data row', 'Index key + Row pointer (or PK)', 'Direct access vs Double lookup'),
('Range Performance', 'Extremely high (Sequential Disk I/O)', 'Lower (Random pointer dereferencing)', 'Date ranges: Clustered wins');

SELECT * FROM comp_q2;
```

OUTPUT / TERMINAL RESULT:

```
mysql>
mysql> SELECT * FROM comp_q2;
```

aspect	clustered_index	non_clustered_index	industry_example
Count per Table	Strictly 1 per table	Multiple secondary indexes allowed	1 Clustered vs N Non-Clustered
Leaf Node Content	Actual complete data row	Index key + Row pointer (or PK)	Direct access vs Double lookup
Physical Storage	Dictates physical record order on disk	Stores key + pointer; data order unchanged	InnoDB PRIMARY KEY vs secondary index
Range Performance	Extremely high (Sequential Disk I/O)	Lower (Random pointer dereferencing)	Date ranges: Clustered wins

```
"=====
```

Question 3

Compare primary indexing and secondary indexing based on storage requirements and search performance.

```
USE dbms_at3;

CREATE TABLE comp_q3 (
    metric          VARCHAR(30) PRIMARY KEY,
    primary_index    VARCHAR(60),
    secondary_index  VARCHAR(60),
    efficiency_factor VARCHAR(80)
);

INSERT INTO comp_q3 VALUES
('Ordering Field',          'Defined on primary ordering key (Sorted)', 'Defined on non-
ordering alternate field',  'Ordering dependency'),
('Sparsity Option',        'Can be Dense or Sparse',          'Must be strictly
Dense',                    'Primary requires fewer index blocks'),
('Storage Footprint',      'Minimal (1 entry per data block)',      'Higher (1 entry per
data record)',             'Primary saves up to 90% space'),
('Search Access Time',     'O(log2 B_index) + 1 Data I/O',          'O(log2 B_index) +
Pointer Resolution',       'Primary avoids extra I/O hops');

SELECT * FROM comp_q3;
```

OUTPUT / TERMINAL RESULT:

```
mysql>
mysql> SELECT * FROM comp_q3;
```

metric	primary_index	secondary_index
Ordering Field	Defined on primary ordering key (Sorted)	Defined on non-ordering alternate field
Search Access Time	O(log2 B_index) + 1 Data I/O	O(log2 B_index) + Pointer Resolution
Sparsity Option	Can be Dense or Sparse	Must be strictly Dense
Storage Footprint	Minimal (1 entry per data block)	Higher (1 entry per data record)

Question 4

Analyze the differences between static hashing and dynamic hashing in database systems.

```
USE dbms_at3;

CREATE TABLE comp_q4 (
    evaluation_criteria VARCHAR(30) PRIMARY KEY,
    static_hashing       VARCHAR(60),
    dynamic_hashing      VARCHAR(60),
    impact_analysis      VARCHAR(80)
);

INSERT INTO comp_q4 VALUES
('Bucket Allocation', 'Fixed number of buckets (M)', 'Dynamically allocated
as data scales', 'Dynamic prevents space wastage'),
('Overflow Resolution', 'Chaining / Linear probing', 'Bucket splitting via
Directory / Modulo', 'Static degrades to linear O(N)'),
('Performance on Growth', 'Degrades drastically with high load', 'Constant O(1)
maintained', 'Dynamic handles arbitrary scale'),
('Reorganization Cost', 'Expensive full table rehashing', 'Incremental per-bucket
split', 'Dynamic ensures zero downtime');

SELECT * FROM comp_q4;
```

OUTPUT / TERMINAL RESULT:

```
mysql>
mysql> SELECT * FROM comp_q4;
+-----+-----+-----+
| evaluation_criteria | static_hashing | dynamic_hashing |
+-----+-----+-----+
| Bucket Allocation | Fixed number of buckets (M) | Dynamically allocated as data scales |
pace wastage |
| Overflow Resolution | Chaining / Linear probing | Bucket splitting via Directory / Modulo |
linear O(N) |
| Performance on Growth | Degrades drastically with high load | Constant O(1) maintained |
bitrary scale |
| Reorganization Cost | Expensive full table rehashing | Incremental per-bucket split |
ro downtime |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

Question 5

Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.

```
USE dbms_at3;

CREATE TABLE comp_q5 (
  architectural_trait  VARCHAR(30) PRIMARY KEY,
  btree_structure      VARCHAR(60),
  bplus_tree_structure VARCHAR(60),
  scale_advantage      VARCHAR(80)
);

INSERT INTO comp_q5 VALUES
('Key & Data Placement', 'Data pointers stored in all nodes',      'Data stored exclusively
in leaf nodes',          'B+ Tree internal nodes hold more keys'),
('Fan-out & Height',      'Lower fan-out, taller tree',            'Maximum fan-out,
shallower tree',          'B+ Tree reduces total disk reads'),
('Sequential Scan',      'Requires recursive in-order traversal', 'Follow linked list along
leaf level',              'B+ Tree provides ultra-fast ranges'),
('Deletion Handling',     'Complex replacement from internal node', 'Simple leaf deletion
with clean merge',        'B+ Tree rebalancing is localized');

SELECT * FROM comp_q5;
```

OUTPUT / TERMINAL RESULT:

architectural_trait	btree_structure	bplus_tree_structure
Deletion Handling is localized	Complex replacement from internal node	Simple leaf deletion with clean merge
Fan-out & Height	Lower fan-out, taller tree	Maximum fan-out, shallower tree
Sequential Scan	Requires recursive in-order traversal	Follow linked list along leaf level
Key & Data Placement	Data pointers stored in all nodes	Data stored exclusively in leaf nodes
Btree structure		

Question 6

Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

```
USE dbms_at3;

CREATE TABLE comp_q6 (
  design_parameter  VARCHAR(30) PRIMARY KEY,
  linear_hashing     VARCHAR(60),
  extendible_hashing VARCHAR(60),
  architectural_impact VARCHAR(80)
);
```

```
);

INSERT INTO comp_q6 VALUES
('Directory Requirement', 'No directory needed (Algorithmic)', 'Requires in-memory directory array', 'Linear saves memory overhead'),
('Split Trigger', 'Linear pointer progression ($P$)', 'Split happens only on overflowing bucket', 'Extendible is demand-driven'),
('Directory Doubling', 'Gradual bucket expansion round-by-round', 'Doubles ($2^d$) when local depth = global', 'Extendible directory may explode'),
('Address Resolution', 'Dual hash functions $h_i(k)$, $h_{i+1}(k)$', 'Global depth bit extraction', 'Linear computes bucket without table');

SELECT * FROM comp_q6;
```

OUTPUT / TERMINAL RESULT:

design_parameter	linear_hashing	extendible_hashing
al_impact		
Address Resolution	Dual hash functions \$h_i(k)\$, \$h_{i+1}(k)\$	Global depth bit extraction
Linear computes bucket without table		
Directory Doubling	Gradual bucket expansion round-by-round	Doubles (\$2^d\$) when local depth = global
Extendible directory may explode		
Directory Requirement	No directory needed (Algorithmic)	Requires in-memory directory array
Linear saves memory overhead		
Split Trigger	Linear pointer progression (\$P\$)	Split happens only on overflowing bucket
Extendible is demand-driven		

Question 7

Compare dense indexing and sparse indexing with respect to memory usage and access speed.

```
USE dbms_at3;

CREATE TABLE comp_q7 (
    comparison_metric    VARCHAR(30) PRIMARY KEY,
    dense_index           VARCHAR(60),
    sparse_index          VARCHAR(60),
    verdict               VARCHAR(80)
);

INSERT INTO comp_q7 VALUES
('Index Entries', 'Exactly 1 entry per record', '1 entry per physical block', 'Sparse uses drastically less space'),
('Memory Footprint', 'Large (may exceed buffer cache)', 'Small (fits easily in main memory)', 'Sparse minimizes RAM saturation'),
('Search Speed (Direct)', 'O(log2 N) direct record pointer', 'O(log2 B) + Scan within block', 'Dense faster if index fits in RAM'),
('File Requirement', 'Applies to sorted and unsorted files', 'Strictly requires physically sorted data', 'Dense offers greater flexibility');
```

```
SELECT * FROM comp_q7;
```

OUTPUT / TERMINAL RESULT:

```

+-----+
| comparison_metric | dense_index | sparse_index |
+-----+-----+-----+
| File Requirement | Applies to sorted and unsorted files | Strictly requires physically sorted data |
| Index Entries | Exactly 1 entry per record | 1 entry per physical block |
| Memory Footprint | Large (may exceed buffer cache) | Small (fits easily in main memory) |
| Search Speed (Direct) | O(log2 N) direct record pointer | O(log2 B) + Scan within block |
+-----+-----+-----+
4 rows in set (0.00 sec)

```

Question 8

Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.

```

USE dbms_at3;

CREATE TABLE comp_q8 (
    feature_dimension VARCHAR(30) PRIMARY KEY,
    indexed_sequential VARCHAR(60),
    direct_file_org VARCHAR(60),
    key_tradeoff VARCHAR(80)
);

INSERT INTO comp_q8 VALUES
('Primary Access Mode', 'Sequential & Indexed B-Tree access', 'Direct mathematical address computation', 'Direct provides pure O(1) access'),
('Range Query Support', 'Fast contiguous range scans', 'Extremely poor (Hash scatters keys)', 'Indexed sequential supports ranges'),
('Storage Overhead', 'Requires storage for index structures', 'Zero index overhead (Hash mapping)', 'Direct saves index memory space'),
('Collision Management', 'Not applicable (Handled by B+ Tree)', 'Requires overflow chains/linear buckets', 'Direct degrades on collisions');

SELECT * FROM comp_q8;

```

OUTPUT / TERMINAL RESULT:

feature_dimension	indexed_sequential	direct_file_org
Collision Management n collisions	Not applicable (Handled by B+ Tree)	Requires overflow chains/linear buckets
Primary Access Mode ure O(1) access	Sequential & Indexed B-Tree access	Direct mathematical address computation
Range Query Support l supports ranges	Fast contiguous range scans	Extremely poor (Hash scatters keys)
Storage Overhead x memory space	Requires storage for index structures	Zero index overhead (Hash mapping)

4 rows in set (0.00 sec)

Question 9

Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

```
USE dbms_at3;

CREATE TABLE comp_q9 (
    structural_aspect      VARCHAR(30) PRIMARY KEY,
    single_level_index     VARCHAR(60),
    multi_level_index      VARCHAR(60),
    retrieval_impact       VARCHAR(80)
);

INSERT INTO comp_q9 VALUES
('Structure Format',      'Single linear flat index file',      'Hierarchical tree
structure (B/B+ Tree)', 'Multi-level scales logarithmically'),
('Binary Search I/O',    'O(log2 B_index) disk block reads',    'O(Height) =
CEIL(log_fanout N) reads',    'Multi-level reduces I/O by ~80%'),
('Large File Scaling',   'Index file becomes too large for RAM',   'Root & intermediate
nodes remain in RAM', 'Multi-level ensures 1-2 disk reads'),
('Insertion Overhead',   'High (Shifting entries in flat file)',   'Low (Node splitting
localized to branch)', 'Multi-level optimizes writes');

SELECT * FROM comp_q9;
```

OUTPUT / TERMINAL RESULT:

```

+-----+-----+-----+
| structural_aspect | single_level_index | multi_level_index |
+-----+-----+-----+
| Binary Search I/O | O(log2 B_index) disk block reads | O(Height) = CEIL(log_fanout N) reads |
| I/O by ~80% | | |
| Insertion Overhead | High (Shifting entries in flat file) | Low (Node splitting localized to branch) |
| es writes | | |
| Large File Scaling | Index file becomes too large for RAM | Root & intermediate nodes remain in RAM |
| 1-2 disk reads | | |
| Structure Format | Single linear flat index file | Hierarchical tree structure (B/B+ Tree) |
| logarithmically | | |
+-----+-----+-----+
4 rows in set (0.00 sec)

```

Question 10

Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

```

USE dbms_at3;

CREATE TABLE comp_q10 (
    query_workload      VARCHAR(30) PRIMARY KEY,
    hashing_performance  VARCHAR(60),
    indexing_performance VARCHAR(60),
    architectural_verdict VARCHAR(80)
);

INSERT INTO comp_q10 VALUES
('Exact Match (Point)', 'O(1) - Single block access', 'O(log_fanout N) - 3 to 4 block reads', 'Hashing wins for key-value lookups'),
('Range Query (BETWEEN)', 'O(N) - Must scan all hash buckets', 'O(log_B N + k) - Contiguous leaf read', 'Tree indexing wins decisively'),
('Ordering (ORDER BY)', 'Requires full filesort O(N log N)', 'Zero cost - Data pre-sorted in index', 'Tree indexing avoids CPU sorts'),
('Prefix Pattern (LIKE)', 'Unsupported (Hash is cryptographic)', 'Supported via B+ Tree left-prefix scan', 'Tree indexing wins for wildcards');

SELECT * FROM comp_q10;

```

OUTPUT / TERMINAL RESULT:

+-----+-----+-----+			
+-----+			
query_workload	hashing_performance	indexing_performance	
+-----+			
Exact Match (Point)	$O(1)$ - Single block access	$O(\log_{\text{fanout}} N)$ - 3 to 4 block reads	
y-value lookups			
Ordering (ORDER BY)	Requires full filesort $O(N \log N)$	Zero cost - Data pre-sorted in index	
s CPU sorts			
Prefix Pattern (LIKE)	Unsupported (Hash is cryptographic)	Supported via B+ Tree left-prefix scan	
for wildcards			
Range Query (BETWEEN)	$O(N)$ - Must scan all hash buckets	$O(\log_B N + k)$ - Contiguous leaf read	
decisively			
+-----+			
4 rows in set (0.00 sec)			